

## Aufgabe 1.4: 2/3

### print\_file.c

```
#include <stdlib.h>
#include <fcntl.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>

#define SIZE 512

int main() {

    int fd = open("./test.txt", O_RDWR);
    if (fd == -1){
        perror("could not open file");
        return EXIT_FAILURE;
    }

    char * buffer = (char *) calloc(SIZE, sizeof(char));

    if (!buffer) {
        perror("could not allocate memory");
        close(fd);
        return EXIT_FAILURE; ✓
    }

    ssize_t r = read(fd, buffer, SIZE-1);

    if (r == -1) {
        perror("could not read file");
        free(buffer);
        close(fd);
        return EXIT_FAILURE;
    }

    printf("%.5s", SIZE-1, buffer);
    free(buffer);
    close(fd);
    return EXIT_SUCCESS;
}
```

## workdir.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

char *mywdfunc(unsigned int bufsize) {
    if (bufsize < 100) {
        return NULL;
    }

    char *buf = (char *)calloc(bufsize, sizeof(char));
    if (buf) {
        char *cwd = getcwd(buf, bufsize-1);
        if (!cwd) {
            free(buf);
            return NULL;
        }
    }

    return buf;
}

int main() {
    char *ret = mywdfunc(512);

    printf("%s", ret);
    free(ret);
    return EXIT_SUCCESS;
}
```

Rückgabewert von mywdfunc() ebenfalls auf NULL prüfen, um Fehlerfall zu erkennen. -1

## Aufgabe 1.5: 6/7

dl\_proc\_list.h 1/1

```
typedef struct dl_proc_node {  
    int pid;  
    int ppid;  
    struct dl_proc_node* prev; ✓  
    struct dl_proc_node* next;  
} dl_proc_node;
```

```
typedef struct dl_proc_list {  
    dl_proc_node* head; ✓  
} dl_proc_list;
```

```
dl_proc_list* dl_proc_list_create(void);
```

```
int dl_proc_list_insert(dl_proc_list *list, int pid);
```

```
int dl_proc_list_get(dl_proc_list *list, int position, int* ✓  
pid, int* ppid);
```

```
int dl_proc_list_remove(dl_proc_list *list, int pid);
```

```
void dl_proc_list_free(dl_proc_list *list);
```

Makefile vorhanden. 0,5/0,5

## dl\_proc\_list.c 3/3,5

```
#include "dl_proc_list.h"
#include <stdlib.h>

// Creates a new process list with the initial process (PID = 1, PPID = 0)
dl_proc_list* dl_proc_list_create() {
    // Create list
    dl_proc_list *list;
    list = (dl_proc_list *) malloc(sizeof(dl_proc_list));

    if (!list) {
        return NULL; ✓
    }

    // Create first process node
    dl_proc_node *first_node;
    first_node = (dl_proc_node *)
malloc(sizeof(dl_proc_node));

    if (!first_node) {
        free(list); ✓
        return NULL;
    }

    // Initialize first process node (PID = 1, PPID = 0)
    first_node->pid = 1;
    first_node->ppid = 0; ✓

    first_node->prev = NULL;
    first_node->next = NULL; ✓

    // Set head of list to first node
    list->head = first_node; ✓

    return list;
}

// Inserts a new process with given PID into process list
// Returns 1 on success, -1 on memory allocation failure,
// -2 on invalid args, -3 if PID already exists
int dl_proc_list_insert(dl_proc_list *list, int pid){
    // Invalid list
    if (!list) {
        return -2; ✓
    }

    // Invalid PID
    if (pid <= 1) {
        return -2; ✓
    }
}
```

```

}

// Find position to insert new process node
dl_proc_node *pre_new = list->head;

while (pre_new && pre_new->next && pre_new->next->pid <
pid) {
    pre_new = pre_new->next;
}

// PID already exists
if ((pre_new && pre_new->pid == pid) ||
    (pre_new && pre_new->next && pre_new->next->pid ==
pid)) {
    return -3; ✓
}

// pre_new is either the end of the list or the last node
with PID < new PID

// Invalid position
if (!pre_new) {
    return -3;
}

// Create new process node
dl_proc_node *new_node = (dl_proc_node *)
malloc(sizeof(dl_proc_node));
if (!new_node) {
    return -1; ✓
}

new_node->pid = pid; ✓

if (pre_new->next) {
    new_node->ppid = pre_new->next->pid; // smaller
} else {
    new_node->ppid = pre_new->pid; // larger inserted
}

new_node->prev = pre_new;
new_node->next = pre_new->next; ✓

if (pre_new->next) {
    pre_new->next->prev = new_node;
}
pre_new->next = new_node; ✓

```

Es können weitere Elemente in der Liste existieren, deren PID größer ist als von pre\_new->next.

inserted before larger  
after smaller  
Liefert nicht notwendigerweise die bisher höchste Prozess-ID. -0.5 ✓

```

    return 0;
}

int dl_proc_list_get(dl_proc_list *list, int position, int*
pid, int* ppid){
    // Invalid list
    if (!list) {
        return -1; ✓
    }

    dl_proc_node *current = list->head;
    int index = 0;

    while (current && index < position) {
        current = current->next;
        index++; ✓
    }

    // Position out of bounds
    if (!current) {
        return -2; ✓
    }

    *pid = current->pid;
    *ppid = current->ppid; ✓

    return 0;
}

```

```

int dl_proc_list_remove(dl_proc_list *list, int pid){
    // Invalid list
    if (!list) {
        return -2; ✓
    }

    // Invalid PID, cannot remove PID 1
    if (pid <= 1) {
        return -2; ✓
    }

    dl_proc_node *current = list->head;

    while (current && current->pid != pid) {
        current = current->next; ✓
    }

    // PID not found
    if (!current) {
        return -3; ✓
    }
}

```

```

}

int parent_pid = current->ppid;

// Remove the node from the list
if (current->prev) {
    current->prev->next = current->next;
} else {
    // Removing head node ✓
    list->head = current->next;
}

if (current->next) {
    current->next->prev = current->prev; ✓
}

free(current); ✓
dl_proc_node *node = list->head;
while (node) {
    if (node->ppid == pid) {
        node->ppid = parent_pid; ✓
    }
    node = node->next;
}

return 0;
}

void dl_proc_list_free(dl_proc_list *list){
    if (!list) {
        return; ✓
    }

    dl_proc_node *current = list->head;
    dl_proc_node *next_node;

    while (current) {
        next_node = current->next;
        free(current); ✓
        current = next_node;
    }

    free(list); ✓
}

```

## proctable.c 1,5/2

```
#include <stdio.h>
#include <stdlib.h>
#include "dl_proc_list.h"

int main(int argc, char *argv[]) {
    dl_proc_list *plist = dl_proc_list_create();
    if (!plist) {
        fprintf(stderr, "Fehler: Konnte Prozesstabelle nicht
erstellen.\n");
        return EXIT_FAILURE; ✓
    }

    for (int i = 1; i < argc; i++) {
        char *endptr;
        long val = strtol(argv[i], &endptr, 10);

        if (*endptr != '\0') {
            fprintf(stderr, "Warnung: Ungueltiges Argument
\"%s\" ignoriert.\n", argv[i]); ✓
            continue;
        }

        if (val == 0) {
            fprintf(stderr, "Warnung: PID = 0 wird ignoriert.
\n");
            continue; ✓
        }

        if (val > 0) {
            if (dl_proc_list_insert(plist, (int)val) < 0) {
                fprintf(stderr, "Fehler beim Einfuegen: PID
%d existiert bereits.\n", val); ✓
            }
        } else {
            int pid_to_remove = (int)(-val);
            if (pid_to_remove == 1) {
                fprintf(stderr, "Fehler: Prozess mit ID = 1
kann nicht geloescht werden.\n"); ✓
            } else if (dl_proc_list_remove(plist,
pid_to_remove) < 0) {
                fprintf(stderr, "Fehler beim Loeschen: Prozess
mit ID = %d existiert nicht.\n", pid_to_remove); ✓
            }
        }
    }

    int pos = 0, pid, ppid;
    while (dl_proc_list_get(plist, pos, &pid, &ppid) == 0) {
        printf("PID = %d, Parent PID = %d\n", pid, ppid); ✓
        pos++;
    }
}
```

strtol() liefert einen Wert vom Typ long zurück. Vor dem Casting nach int darauf achten, dass der Wert nicht unter INT\_MIN bzw. über INT\_MAX liegt. -0,5



```
}  
    dl_proc_list_free(plist);  
    return EXIT_SUCCESS; ✓  
}
```